

GENOSIGHT

AI-Based Genetic Variant Pathogenicity Prediction System

Project Documentation

Group Members:	Alishba Shahzad	F24607022
	Husnain Haider	F24607004

Purpose	Predict whether a genomic variant is benign or pathogenic using a trained XGBoost classifier, with supporting gene and disease association lookup.
Primary Model	XGBoost classifier trained on seven engineered genomic / annotation-derived features.
Application	Streamlit web interface for interactive single-variant analysis.
Data Source	ClinVar VCF data and derived lookup artifacts.
Scope	Research / educational prototype; not a clinically validated diagnostic system.

Document note. This document is grounded in the supplied training notebook, evaluation outputs, requirements file, and deployed app.py. Statements about model performance describe the documented experiments rather than clinical performance.

GenoSight is best characterized as a variant pathogenicity prediction and annotation-lookup prototype. The deployed classifier predicts the pathogenicity class; the displayed disease associations are retrieved from the project's lookup table.

Contents

1. Executive Summary
2. System Scope and Objectives
3. High-Level Architecture
4. Data Source and Data Processing
5. Feature Engineering
6. Model Development and Selection
7. Evaluation Results
8. Deployment Architecture and Application Workflow
9. Model Artifacts and Project Structure
10. Installation and Reproduction
11. Technical Limitations and Risks
12. Future Development
13. Conclusion

Core implementation

Component	Implementation
Programming language	Python
Web framework	Streamlit
Data processing	Pandas + NumPy
ML / preprocessing	Scikit-learn
Final classifier	XGBoost
Serialization	Joblib
Input source	ClinVar VCF
Deployment inputs	Chromosome, position, REF, ALT
Prediction classes	Benign / Pathogenic

1. Executive Summary

GenoSight is an end-to-end machine-learning prototype for genomic variant pathogenicity prediction. The training pipeline begins with ClinVar VCF data, extracts clinical significance, gene, and disease annotations, cleans and filters records, constructs a balanced binary classification dataset, engineers seven numerical features, and evaluates multiple classifiers. XGBoost is used as the deployed model because it produced the strongest observed results among the tested model families in the documented experiments.

The deployed application is implemented in Streamlit. A user supplies a chromosome, genomic position, reference allele, and alternate allele. The application encodes the inputs using the same saved mappings used during training, retrieves gene/disease metadata from a lookup table, applies the saved StandardScaler, and passes the resulting seven-feature vector to the trained XGBoost classifier. The interface then displays the predicted class, prediction probabilities, and available gene/disease associations.

The strongest documented XGBoost experiment reached approximately 77% accuracy on the held-out test split used by the notebook. This value is an experimental machine-learning metric; it is not a measure of clinical diagnostic accuracy.

2. System Scope and Objectives

2.1 Problem Statement

The system is designed to classify individual genomic variants into two pathogenicity classes based on their ClinVar clinical-significance annotations: **pathogenic-class** and **benign-class**. The model is trained using genomic variant and annotation-derived features and is deployed through a Streamlit application for single-variant analysis.

The system is **not a patient-level disease diagnosis system**. It does not use symptoms, medical history, phenotype, or other patient-specific clinical information. Disease and gene names displayed by the application are retrieved from the project's annotation lookup table rather than directly predicted by the classifier.

2.2 Objectives

- Process large-scale genomic variant records from a ClinVar VCF source.
- Extract clinically relevant annotation fields from VCF INFO entries.
- Convert supported clinical significance categories into a binary target.
- Construct a balanced training dataset.
- Engineer a compact seven-feature numerical representation.
- Compare Random Forest, Gradient Boosting, and XGBoost classifiers.
- Serialize the final model and preprocessing artifacts.
- Deploy an interactive single-variant analysis interface.
- Retrieve available gene and disease associations for presentation alongside the prediction.

2.3 Scope boundary:

What the model predicts: a binary pathogenicity class for the supplied variant representation.

What the application additionally provides: annotation lookup from the project's precomputed disease table.

What it does not establish: a clinical diagnosis, treatment recommendation, or proof that a disease association inferred from a nearby lookup record applies to the exact input variant.

3. High-Level Architecture

3.1 Training pipeline

1. ClinVar VCF	→	2. Annotation extraction	→	3. Cleaning & filtering
4. Binary labels	→	5. Balancing + feature engineering	→	6. Train / test split
7. StandardScaler	→	8. Model comparison	→	9. Saved artifacts

3.2 Deployment pipeline

User input	→	Encoding + lookup	→	Seven-feature vector
Saved StandardScaler	→	Saved XGBoost model	→	Streamlit result

4. Data Source and Data Processing

4.1 Source dataset

The project uses ClinVar VCF data containing genomic variant records and associated annotations. A documented parsing run produced approximately 4,122,274 imported records before final filtering. The VCF fields are initially represented as CHROM, POS, ID, REF, ALT, QUAL, FILTER, and INFO.

4.2 Annotation extraction

```
Clinical_Significance = INFO → CLNSIG  
Gene = INFO → GENEINFO  
Disease_Name = INFO → CLNDN
```

The INFO column is a compound annotation field. The notebook extracts the fields above using regular-expression patterns. Records missing the required clinical significance, gene, or disease values are removed before the binary classification dataset is formed.

4.3 Variant cleaning

- The workflow focuses on single-nucleotide REF/ALT alleles from the A/C/G/T alphabet.
- Human chromosome representations are mapped for chromosomes 1-22 plus X, MT, and Y.
- Unsupported or non-standard contigs such as NT_187693.1 and NT_187661.1 were observed during processing and require careful handling.
- Intermediate notebook executions reported slightly different balanced dataset sizes because the notebook was run multiple times with different intermediate states and filtering outcomes.

4.4 Target construction

Clinical significance category	Binary target
Pathogenic	1
Likely pathogenic	1
Pathogenic/Likely pathogenic	1
Benign	0
Likely benign	0
Benign/Likely benign	0
Uncertain / unsupported	Excluded from binary task

4.5 Class balancing

The notebook samples records so that the final training data contain equal numbers of benign and pathogenic examples. One documented execution produced 345,064 balanced records: 172,532 pathogenic and 172,532 benign.

5. Feature Engineering

The final classifier receives seven features. The feature order is part of the model contract and is reproduced explicitly in the deployed application before scaling and inference.

Feature	Type / construction	Purpose
REF_encoded	A/C/G/T → 0/1/2/3	Reference allele
ALT_encoded	A/C/G/T → 0/1/2/3	Alternate allele
CHROM_encoded	1-22, X, MT, Y → integer	Chromosome representation
POS	Integer genomic coordinate	Variant position
Gene_encoded	LabelEncoder-derived integer	Encoded gene identity
Num_Diseases	Count from pipe-delimited names	Number of listed associations
Disease_Unknown	Binary indicator	Unavailable / unspecified annotation

5.1 Nucleotide encoding

```
A → 0
C → 1
G → 2
T → 3
```

The mapping is serialized as `nuc_map.pkl` and loaded by the application so that inference matches training.

5.2 Chromosome encoding

```
1-22 → 1-22
X → 23
MT → 24
Y → 25
```

The mapping is serialized as `chrom_order.pkl`. The deployed application deliberately uses this saved dictionary rather than reconstructing the mapping independently.

5.3 Gene / disease-derived features

Gene names are encoded numerically, while disease information is converted into two compact features: the number of listed diseases and a flag indicating unavailable disease annotation. These values are recovered from the project's lookup table at inference time.

6. Model Development and Selection

6.1 Train/test split

The dataset was divided into training and testing subsets using `train_test_split` with a test size of **1%**, `random_state=42`, and `stratify=y`. The relatively small test fraction was selected because the dataset contained hundreds of thousands of balanced records; consequently, 1% still provided several thousand held-out samples for evaluation while retaining the majority of the data for model training.

Stratified sampling was used to preserve the proportion of the two target classes—benign and pathogenic—in both the training and test subsets. This helps ensure that the evaluation set represents the class distribution of the balanced dataset.

Although this approach provided a sufficiently large test set for the current prototype, a larger hold-out set and/or cross-validation would provide stronger evidence of model generalization in future research.

6.2 Scaling

A `StandardScaler` is fit on the training data and then applied to the test data. The fitted scaler is saved as `scaler.pkl` and reused during application inference. This ensures that the deployment feature vector is transformed using the same learned parameters as training.

6.3 Candidate models

Model family	Observed result in supplied experiments	Deployment
Random Forest	~70.9% best shown accuracy	Not deployed
Gradient Boosting	~68.1% best shown accuracy	Not deployed
XGBoost	~76.9% best shown accuracy	Selected / deployed

6.4 XGBoost configuration

```
XGBClassifier(  
    n_estimators=890,  
    random_state=42  
)
```

XGBoost was selected as the final machine-learning model. It was configured with 890 trees and a fixed random state of 42. Different numbers of trees were tested during experimentation, and the highest observed accuracy was approximately 76.9%. The final trained model was saved as `xgboost_dna_model.pkl`.

6.5 Implementation contract

The deployed application expects the exact seven-feature order, the saved chromosome and nucleotide encoders, the saved scaler, and the saved model. A mismatch in any of these preprocessing components can change the model input distribution and invalidate inference.

7. Evaluation Results

7.1 Best documented XGBoost classification report

Class	Precision	Recall	F1-score
Benign (0)	0.77	0.76	0.77
Pathogenic (1)	0.76	0.77	0.77
Overall accuracy	0.77	-	-

7.2 Interpretation

The model gave similar results for both classes in terms of precision, recall, and F1-score. XGBoost performed better than the other tested models, so it was selected as the final model for the project.

7.3 Evaluation Limitations

- The accuracy is based on the test data used in the notebook. It does not represent clinical accuracy.
- Only 1% of the balanced dataset was used for testing.
- The model was not tested on a separate external dataset.
- Some genomic records may be repeated or very similar, which could make the results look better than they actually are.
- The model's probability scores have not been tested to see how reliable they are as real-world clinical probabilities.

8. Deployment Architecture and Application Workflow

8.1 Application stack

Layer	Implementation	Responsibility
Presentation	Streamlit + custom HTML/CSS	Collect variant input and render results
Application logic	Python in app.py	Validation, encoding, lookup, feature construction
Preprocessing	Saved StandardScaler	Training-time feature transformation
Model	Saved XGBoost classifier	Class and probability prediction
Annotation	disease_lookup.csv	Gene/disease lookup
Serialization	Joblib	Load model and preprocessing assets

8.2 Asset loading

```
model = joblib.load("xgboost_dna_model.pkl")
nuc_map = joblib.load("nuc_map.pkl")
chrom_order = joblib.load("chrom_order.pkl")
scaler = joblib.load("scaler.pkl")
lookup = pd.read_csv("disease_lookup.csv")
```

The application uses Streamlit resource caching so the model and lookup assets are not repeatedly loaded for every interaction.

8.3 Input validation

The interface restricts chromosome selection to 1-22, X, Y, and MT; nucleotide selectors to A/T/G/C; and position to a positive integer. The application also warns when REF and ALT are identical.

8.4 Exact lookup and nearest-variant fallback

The application first searches the lookup table for an exact CHROM_encoded + POS match. If no disease associations are available, it searches the same chromosome for the nearest known position within a 50,000-base window. This is an annotation convenience feature, not a biological inference engine.

8.5 Inference

```
X_raw = [REF_enc, ALT_enc, CHROM_enc, POS, Gene_enc, Num_Diseases, Disease_Unknown]
X_input = scaler.transform(X_raw)
pred = model.predict(X_input)
proba = model.predict_proba(X_input)
```

The application displays the predicted class and probability distribution. The displayed 'confidence' is the highest model class probability; it is not a clinically validated confidence interval or calibrated disease risk.

9. Model Artifacts and Project Structure

9.1 Runtime artifacts

File	Type	Role
xgboost_dna_model.pkl	Joblib / PKL	Trained XGBoost classifier
nuc_map.pkl	Joblib / PKL	Nucleotide encoding dictionary
chrom_order.pkl	Joblib / PKL	Chromosome encoding dictionary
scaler.pkl	Joblib / PKL	Fitted StandardScaler
disease_lookup.csv	CSV	Gene/disease/feature lookup table

9.2 Project structure

```

ICAT-2-AI-Based-Genetic-Disease-Detection-System/
|-- app.py
|-- temporary/
|   |-- main.ipynb
|-- clinvar.vcf
|-- disease_lookup.csv
|-- xgboost_dna_model.pkl
|-- scaler.pkl
|-- nuc_map.pkl
|-- chrom_order.pkl
|-- requirements.txt
|-- README.md

```

9.3 Dependency set

Package	Version specified	Purpose
streamlit	1.55.0	Web application
pandas	2.3.3	Data manipulation
numpy	2.4.2	Numerical arrays
scikit-learn	1.8.0	Preprocessing, evaluation, classical ML
joblib	1.5.3	Artifact serialization
xgboost	3.2.0	Final classifier

10. Installation and Reproduction

10.1 Environment setup

```
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
```

10.2 Launch the application

```
streamlit run app.py
```

10.3 Runtime file requirements

For inference, the following files must be in the expected working directory: `xgboost_dna_model.pkl`, `nuc_map.pkl`, `chrom_order.pkl`, `scaler.pkl`, and `disease_lookup.csv`.

10.4 Training reproduction

The notebook `main.ipynb` contains the training and experimentation workflow. The complete VCF is large, and full-file Pandas parsing can be memory-intensive. A sufficiently provisioned environment, or chunked / staged processing, is therefore recommended when reproducing the import stage.

10.5 Inference sequence

```
User Input
↓
Encode REF / ALT / CHROM
↓
Lookup gene + disease-derived features
↓
Build 7-feature vector
↓
scaler.transform(...)
↓
model.predict(...) / model.predict_proba(...)
↓
Render result
```

11. Technical Limitations and Risks

11.1 Modeling limitations

- **Limited features.** The model uses only seven features, so it does not use all the information that can be available for genetic variant analysis.
- **Gene encoding.** Genes are converted into numbers using LabelEncoder. These numbers are only labels and do not show any biological relationship between genes.
- **Genomic position.** The POS value is treated as a number. Two variants that are close to each other in position are not necessarily related in function.
- **Feature scaling.** StandardScaler is used in the implemented pipeline. However, XGBoost does not normally require feature scaling because it is a tree-based model.
- **Evaluation design.** Only 1% of the data was used for testing. Although this still provides thousands of test samples, a larger test set could provide stronger evaluation results.
- **Generalization.** The model was not tested on a separate external dataset, so its performance on completely new data is not confirmed.

11.2 Data integrity and leakage considerations

- **Duplicate records.** ClinVar may contain repeated or very similar records for the same genetic location. These records should be checked and cleaned in future versions.
- **Data leakage.** The data is split row by row, so very similar variants could appear in both the training and testing data. Grouping related variants before splitting would give a more reliable evaluation.

11.3 Annotation lookup limitation

The nearest-variant fallback uses physical genomic distance to recover annotation data. This can improve the user experience, but it must not be interpreted as a biological claim that the nearby variant and the queried variant share the same disease association.

11.4 Clinical-use limitation

The system is not clinically validated and should not be used independently for diagnosis, treatment, genetic counseling, or other clinical decision-making. Its model probability should not be equated with disease penetrance, absolute risk, or medical certainty.

12. Future Development

The current implementation provides a useful engineering baseline. The following improvements would strengthen its scientific rigor, usability, and research value.

Priority area	Recommended development
Input handling	Allow users to upload VCF files and analyze multiple variants at once instead of entering one variant manually.
Feature engineering	Add more useful genetic features, such as DNA sequence information, conservation, allele frequency, and functional annotations.
Categorical representation	Replace simple gene numbers with a better way of representing genes that preserves meaningful biological information.
Evaluation	Use cross-validation, larger test sets, and separate external datasets to better evaluate the model.
Data splitting	Group similar or related variants before splitting the data to reduce data leakage and get more realistic results.
Metrics	Use more evaluation measures, such as ROC-AUC, PR-AUC, confusion matrix, sensitivity, specificity, and calibration.
Explainability	Add SHAP or similar methods to show why the model made a particular prediction.
Annotation	Use authoritative, versioned annotation resources for transparent provenance.
Clinical safety	Add explicit research-use notices, provenance tracking, model versioning, and audit logs.
Reproducibility	Convert notebook logic into a deterministic training script / pipeline with configuration and versioned data.

13. Conclusion

GenoSight demonstrates a complete machine-learning lifecycle for genomic variant pathogenicity classification: large-scale VCF processing, annotation extraction, data cleaning, class balancing, feature engineering, model comparison, model serialization, and interactive deployment.

The implementation's strongest documented model is XGBoost, with an observed accuracy of approximately 77% in the supplied evaluation. The Streamlit application preserves the training-time preprocessing contract through saved encoders and scaler artifacts and augments the prediction with gene/disease lookup information.

From a research engineering perspective, the project provides a solid prototype foundation. The highest-value next steps are rigorous data splitting, external validation, richer genomic features, explainability, and a more deterministic training pipeline. Those improvements would be required before the system could support claims beyond research and educational use.

Final technical characterization

System type	AI-assisted genomic variant pathogenicity prediction prototype
Model	XGBoost classifier
Features	7 engineered genomic / annotation-derived features
Deployment	Streamlit
Primary output	Benign / Pathogenic + probability breakdown
Secondary output	Gene and disease association lookup
Research status	Prototype; not clinically validated